Kubernetes API Access for Mulat / Postman Testing

Date: 2026-11-04

This guide explains how to expose the Kubernetes API server for external clients (such as Postman), create a read-only ServiceAccount token aligned with Tool_API.md, and verify access. Manifests live under `deploy/kubernetes/` in the mulat project.

Target cluster example: single control-plane node (`kube-apiserver-node1` in `kube-system`), Calico, CoreDNS.

Overview

Goal	Approach
Authenticate as read-only caller	ServiceAccount `mulat-reader` in namespace `ai-agents`
Reach API from a laptop	Node IP `:6443`, or NodePort `:30443`, or in-cluster `kubernetes` Service
Test HTTP calls	Postman collection `postman/mulat-agent-api.postman_collection.json`

The API server runs as a static pod on the control plane. Inside the cluster, workloads use the built-in Service `kubernetes` in namespace `default` at `https://kubernetes.default.svc:443`. That ClusterIP is not reachable from your PC unless you port-forward or use the node address.

Step 1 — Apply read-only RBAC

Apply the namespace and RBAC manifests:

```
kubectl apply -f deploy/kubernetes/namespace.yaml
kubectl apply -f deploy/kubernetes/mulat-readonly-rbac.yaml
```

Resources created:

- **Namespace:** `ai-agents`
- **ServiceAccount:** `mulat-reader`
- **ClusterRole / ClusterRoleBinding:** `mulat-reader`

Permissions match Tool_API.md:

- Core API: `nodes`, `pods`, `events`, `namespaces` — verbs `get`, `list`, `watch`
- Metrics API: `nodes`, `pods` in group `metrics.k8s.io` — verbs `get`, `list`

No access to `secrets`, `pods/exec`, `pods/log`, or write verbs.

Verify:

```
kubectl auth can-i list nodes --as=system:serviceaccount:ai-agents:mulat-reader
kubectl auth can-i list pods --as=system:serviceaccount:ai-agents:mulat-reader
```

Both should answer `yes`.

Step 2 — Get a bearer token

On Kubernetes 1.24 and later, use bound ServiceAccount tokens (recommended):

```
kubectl create token mulat-reader -n ai-agents --duration=24h
```

Copy the entire JWT string. In Postman, set environment variable `k8sToken` to that value.

Longer lifetime (example: 30 days):

```
kubectl create token mulat-reader -n ai-agents --duration=720h
```

Re-run the command when the token expires. Do not commit tokens to git.

Step 3 — Expose the API server

Choose one method depending on network layout.

Option A — Direct HTTPS on port 6443 (preferred when open)

On kubeadm-style clusters, the API server usually listens on the control-plane node at **6443**.

On the node:

```
ss -lntp | grep 6443
```

From a client (replace host with your node IP or DNS name):

```
curl -k -H "Authorization: Bearer TOKEN" \
  "https://CLUSTER264VM:6443/api/v1/nodes?limit=1"
```

Postman environment:

- `k8sServer` = `https://CLUSTER264VM:6443` (or `https://<node-ip>:6443`)
- Disable SSL certificate verification in Postman if the apiserver uses a self-signed or private CA certificate.

Option B — NodePort Service on port 30443

Use when port 6443 is not reachable from outside but node IPs are.

Apply:

```
kubectl apply -f deploy/kubernetes/apiserver-nodeport-service.yaml
kubectl get svc kube-apiserver-external -n kube-system
kubectl get endpoints kube-apiserver-external -n kube-system
```

Service summary:

- **Name:** `kube-apiserver-external` (namespace `kube-system`)
- **Type:** NodePort
- **nodePort:** 30443 → **targetPort:** 6443 on apiserver pods
- **Selector:** `component=kube-apiserver`, `tier=control-plane`

Postman: `k8sServer` = `https://<any-node-ip>:30443`

If **Endpoints** are empty, inspect apiserver pod labels and update the Service selector:

```
kubectl get pod -n kube-system kube-apiserver-node1 --show-labels
```

Security: NodePort exposes the API on every node's IP. Restrict with firewall rules or VPN.

Option C — In-cluster only

```
kubectl get svc kubernetes -n default
```

Use `https://kubernetes.default.svc` from pods inside the cluster. For local Postman, combine with `kubectl port-forward` to the apiserver or use Option A or B.

Step 4 — Postman setup

1. Import `postman/mulat-agent-api.postman_collection.json` and `postman/mulat-local.postman_environment.json`. 2. Select environment **Mulat — Local**. 3. Set variables:

Variable	Example
`k8sServer`	`https://192.168.1.10:6443` or `https://192.168.1.10:30443`
`k8sToken`	JWT from `kubectl create token`
`prometheusUrl`	`http://<prometheus-host>:9090` (for right-sizing tool queries)
`baseUrl`	`http://localhost:8000` (mulat FastAPI, when deployed)

4. Open folder **Tool backing APIs (read-only, manual)** → **list_nodes — GET /api/v1/nodes** → Send.

Expected: HTTP 200 and JSON listing nodes. HTTP 401 means wrong or expired token. HTTP 403 means RBAC denies the action.

Agent API folder (`POST /ask`, `/alert`, `/approve`) requires the mulat FastAPI service on `baseUrl`.

Step 5 — Quick curl reference

List nodes:

```
curl -k -H "Authorization: Bearer TOKEN" \
  "${K8S_SERVER}/api/v1/nodes?limit=5"
```

List pods in all namespaces:

```
curl -k -H "Authorization: Bearer TOKEN" \
  "${K8S_SERVER}/api/v1/pods?limit=10"
```

Set `K8S_SERVER` to the same URL as Postman `k8sServer` (no trailing slash).

Manifest file reference

File	Purpose
`deploy/kubernetes/namespace.yaml`	Creates `ai-agents` namespace
`deploy/kubernetes/mulat-readonly-rbac.yaml`	ServiceAccount, ClusterRole, ClusterRoleBinding
`deploy/kubernetes/apiserver-nodeport-service.yaml`	Optional NodePort 30443 to apiserver

Related documentation:

- `Tool_API.md` — tool contracts and RBAC rationale
- `postman/mulat-agent-api.postman_collection.json` — HTTP requests for agents and backing APIs

Security and operations notes

- Treat `mulat-reader` tokens like passwords; rotate with `kubectl create token` on a schedule.
- NodePort exposure increases attack surface; prefer direct 6443 with strict firewall or bastion access when possible.
- Read-only RBAC does not prevent reading all pod specs and events across the cluster; scope namespaces in application logic if you add that later.

• Optional metrics in Tool_API (`describe_node` usage) require metrics-server and the `metrics.k8s.io` RBAC rule already included.

• Control-plane pod restarts (scheduler, controller-manager) are unrelated to API exposure; troubleshoot with `kubectl logs` in `kube-system` and node system logs if the API becomes unavailable.

Troubleshooting

Symptom	Likely cause	Action
Connection refused on 6443	Firewall or apiserver not bound to client-facing IP	Check `ss -lntp`, security groups, iptables
Connection refused on 30443	NodePort Service missing or no endpoints	Apply Service manifest; fix selector labels
HTTP 401	Missing or invalid token	Run `kubectl create token` again
HTTP 403	RBAC too narrow or wrong SA	Re-check ClusterRoleBinding and `kubectl auth can-i`
SSL errors in Postman	Untrusted apiserver cert	Use `-k` in curl or disable SSL verify in Postman for lab only
Empty node list with 200	RBAC allows list but cluster has no worker nodes registered	`kubectl get nodes` from admin kubeconfig

---

Related cluster components (example)

Typical `kube-system` pods on a lab cluster:

- `kube-apiserver-node1` — API server static pod
- `kube-controller-manager-node1`, `kube-scheduler-node1` — control plane
- `calico-node-*`, `coredns-*` — networking and DNS

The API access steps above do not modify these components; they add RBAC and an optional Service for external HTTPS access.